SOA

Diseño de sistemas

1. Introducción

Los sistemas de información han recorrido durante su historia un camino que lleva desde los primitivos sistemas monolíticos, pasando por sistemas cliente/servidor, hasta llegar a la Arquitecturas Orientada a Servicios (SOA), en la que la funcionalidad de las aplicaciones se implementa bajo la forma de componentes reutilizables, que se invocan a través de interfaces estándar. Estos componentes –llamados *servicios*– pueden combinarse para crear otros servicios más complejos. Hoy en día toda la industria, tanto los proveedores de tecnología como los consultores y los usuarios, están de acuerdo en que SOA permitirá mejorar la calidad de las aplicaciones y dar una mejor respuesta a las necesidades de los negocios. En los próximos párrafos explicaremos por qué.

Una de las principales necesidades que enfrentan hoy las empresas es la de integrar sus procesos en forma "transversal" a través de las diferentes aplicaciones, y ésta es precisamente una necesidad a la cual las arquitecturas IT anteriores casi nunca pudieron dar una respuesta completamente satisfactoria. Tradicionalmente la integración de los procesos se llevó a cabo, en algunos casos manualmente, obligando a los usuarios a conectarse a diferentes aplicaciones (y en muchos de esos casos a utilizar un *sign-on* diferente cada vez), o a llevar papeles de un sector de la empresa a otro, con las consiguientes inexactitudes e ineficiencias; en otros casos la lógica de integración de los procesos estaba embebida dentro del código de las aplicaciones, lo que torna complicado poder modificar ágilmente los procesos, o crear procesos nuevos reutilizando los componentes existentes.

En el mundo de los negocios de hoy, las empresas necesitan tener una gran flexibilidad para poder adaptarse ágilmente a lo que un entorno muy exigente les demanda. Una empresa tendrá esa flexibilidad sólo si sus procesos de negocio están integrados punta a punta, tanto internamente como con sus socios, clientes y proveedores, y si pueden ser modificados rápidamente para responder a cualquier demanda de los clientes, oportunidad del mercado o amenaza externa. Las empresas que logren esto cubrirán mejor las demandas de sus clientes, ganarán competitividad e incrementarán en definitiva sus beneficios.

Como ya hemos comentado antes, el problema de las arquitecturas tradicionales reside en que, en ellas, las aplicaciones no están diseñadas para ser integradas. En esas arquitecturas las aplicaciones se diseñan generalmente como "silos verticales", cada una de ellas pensada para un propósito específico y un alcance limitado. En muchos casos se trata de aplicaciones construídas o adquiridas en diferentes momentos históricos, por diferentes equipos de gente y en forma independiente, de modo que es natural que su integración resulte dificultosa.

Este estado de cosas es obviamente indeseable ya que la infraestructura tecnológica no debería inhibir sino, por el contrario, facilitar la integración de las aplicaciones y de los procesos de negocio, permitiendo además que dichos procesos reutilicen las aplicaciones existentes e incorporen también a las nuevas aplicaciones que se desarrollen en el futuro, incluso sobre diferentes plataformas. La arquitectura orientada a servicios (SOA) permite justamente crear una infraestructura tecnológica capaz de brindar la agilidad que los negocios necesitan en el mundo de hoy.

2. La Arquitectura Orientada a Servicios (SOA)

Tal como lo comentamos en el apartado anterior, existe hoy en día un amplio consenso en que la mejor respuesta para las necesidades actuales de las empresas es la ***Arquitectura Orientada a Servicios (SOA)***. Pero, ¿qué es exactamente SOA?

SOA es una manera de diseñar y construir aplicaciones de negocio en la cual la funcionalidad aplicativa se provee a través de componentes funcionales denominados ***servicios***. Cada servicio representa una función de negocios de la empresa tal como: obtener la información de un cliente, dar de baja una línea telefónica, dar de alta un nuevo empleado, etc. Los servicios se comunican unos con otros a través de interfaces bien definidas, estándar, e independientes de la plataforma de hardware, sistema operativo, o lenguaje de programación en el que el servicio esté implementado. Esto permite crear ***aplicaciones compuestas*** (en inglés *composite applications*) simplemente combinando o ensamblando servicios más básicos. Las aplicaciones compuestas son también servicios, pero de más alto nivel.

Los servicios, entonces, se comunican mediante interfaces estándar, y no ligadas a ninguna implementación particular. Dicho en otras palabras, se comportan como cajas negras, en las que están expuestas las interfaces (es decir: la forma de invocarlos y los resultados que producen), pero no la implementación interna. Esta característica se conoce como ***acoplamiento débil*** entre servicios. El beneficio del acoplamiento débil consiste en la mayor independencia que provee entre los servicios; las modificaciones hechas en un servicio no impactan en los demás. Esto a su vez brinda mayor agilidad para implementar cambios y permite ahorrar costos de desarrollo y mantenimiento de los sistemas. El acoplamiento fuerte por el contrario, implica que los diferentes componentes de una aplicación estén íntimamente relacionados en funcionalidad y forma, lo cual los hace muy vulnerables a los cambios evolutivos, ya que cualquier modificación a uno de sus componentes termina afectando a los demás.

La necesidad de contar con sistemas débilmente acoplados surge de la necesidad ya mencionada de dotar a las aplicaciones de mayor agilidad, lo cual se basa a su vez en la necesidad del negocio de adaptarse rápidamente a su ambiente cambiante. Podría decirse que, en última instancia, el propósito de SOA es desvincular las aplicaciones de las implementaciones de los componentes que dichas aplicaciones utilizan. A esto se lo llama ***"separación de las incumbencias"*** *("separation of concerns"* en inglés). La gran ventaja de esta separación es que permite cambiar la implementación de los componentes sin afectar a las aplicaciones y, viceversa, modificar las aplicaciones

reutilizando los mismos componentes. Es evidente que este modelo puede darle a los negocios la flexibilidad que los sistemas tradicionales no podían brindar.

Muchas de las ideas de SOA no son totalmente nuevas. Sin embargo una característica que diferencia a SOA de otras arquitecturas similares del pasado es la

importancia que ha adquirido en SOA un avance relativamente reciente, que aprovecha la amplia aceptación del XML como lenguaje común de descripción de datos y que consiste en utilizar para describir los servicios y sus interfaces un conjunto de especificaciones basadas en XML. Estas especificaciones son llamadas ***Web Services***.

No deben confundirse sin embargo los planos: SOA es una arquitectura, una forma de diseñar y construir sistemas, mientras que los Web Services son una forma de implementar servicios en una SOA, y no necesariamente la única. Por ejemplo, un middleware orientado a mensajería puede brindar una forma válida de implementar servicios. Por otra parte un modelo de arquitectura necesita mucho más que una especificación de servicios. Necesita definir por ejemplo la lógica que utiliza una aplicación compuesta para invocar a los diferentes servicios en una determinada secuencia. Esto significa ni más ni menos que relacionar los procesos comerciales de un negocio con sus procesos técnicos, ya que una aplicación compuesta representa en realidad un proceso de negocio. Por ejemplo, el pago a proveedores es un proceso de negocio que en SOA se implementaría mediante una aplicación compuesta, mientras que actualizar la base de datos de partes para reflejar la recepción de una nueva remesa de mercaderías es un servicio, que se implementará como un componente de software (o sea, un proceso técnico), y que será invocado por el proceso de pago a proveedores en alguna parte de su lógica. Vemos entonces que los procesos de negocio juegan un rol muy significativo en SOA.

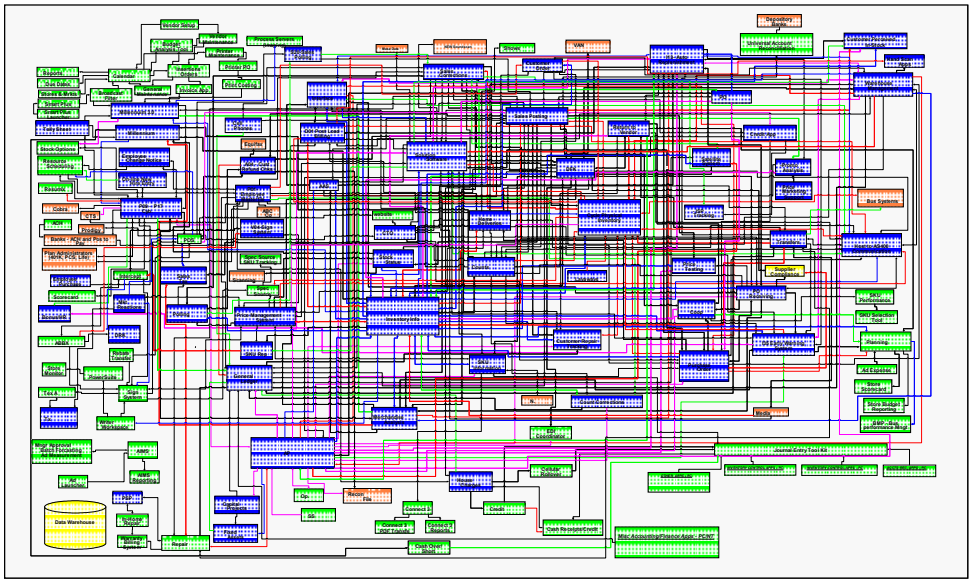
Yendo aún un paso más allá, un proceso de negocios puede incluir operaciones (servicios) no sólo internos, entre los departamentos de la propia empresa, sino también con asociados de negocio externos, sobre los que la empresa no tiene ningún control. Esto hace necesario en una SOA definir políticas operativas y acuerdos de niveles de servicios para definir la forma en que las empresas van a interactuar. Finalmente todo esto tiene que operar en un ambiente confiable y seguro para que los procesos puedan llevarse a cabo en los términos acordados. Por lo tanto la seguridad y la confiabilidad también tienen un rol significativo en SOA.

3. Entrerprise Service Bus (ESB)

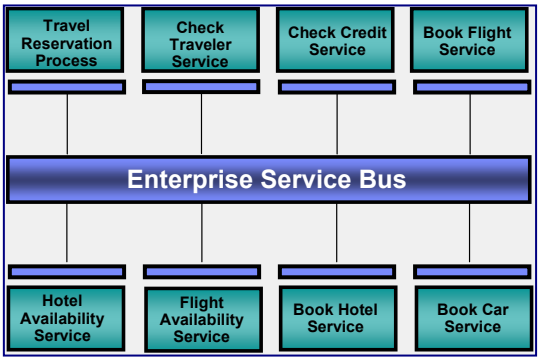
En SOA los diferentes servicios habitualmente no interactúan en forma directa unos con otros sino que lo hacen utilizando un ***Enterprise Service Bus (ESB)***. El ESB es un patrón *(pattern)* de diseño, que representa una buena práctica de arquitectura, y que recomienda no interconectar los servicios directamente –punta a punta– sino, por el

contrario, utilizar un *bus* de interconexión, que habitualmente se implementa con un middleware específicamente diseñado para tal fin.

La recomendación de no interconectar los servicios punta a punta tiene como objeto evitar una situación tal como la que muestra la siguiente figura, que es el mapa de las interfaces entre aplicativos tomado de un caso real (una empresa de productos electrónicos de consumo masivo):



Un ESB provee una infraestructura de conectividad para los diferentes servicios a través de la cual fluye la información que permite que aquellos interactúen, de modo que la arquitectura de integración de los servicios pasa a convertirse en algo como esto:



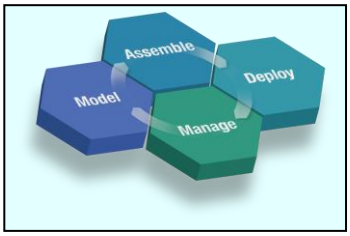
Un ESB no se limita a interconectar los diferentes servicios sino que provee además una ***mediación*** inteligente entre lo servicios. El concepto de mediación incluye:

- El ***ruteo*** de cada requerimiento al componente o a los componentes correspondientes, en base al contenido del requerimiento (*content-based routing*). En algunos casos un servicio se resolverá con único componente, pero puede haber otros casos en los que el ESB deba invocar a una secuencia de componentes (Ej: transacciones en un mainframe, o en SAP) para cumplir con un servicio determinado.
- El ***reformateo*** de los datos para adaptarlos a los diferentes servicios participantes; por ejemplo, de archivo plano a XML.
- El ***cambio de protocolo*** de transporte (por ejemplo, de JMS a HTTP).

El ESB implementa entonces la lógica de integración de los servicios, no la lógica de negocios que está íntegramente implementada en los componentes. Se puede decir también que el ESB es el lugar donde los servicios se "publican". Cada vez que una aplicación o un proceso necesiten invocar un servicio se lo va a "pedir" al ESB y éste se va a encargar de mediar entre quien requirió el servicio y quien lo puede brindar realmente.

4. Los procesos de negocio: BPM basado en SOA

Como se dijo más arriba, en SOA es muy importante la capacidad para combinar servicios para crear aplicaciones compuestas que llevan a cabo procesos de negocios. Esto es central en SOA ya que está directamente asociado al objetivo de darle mayor flexibilidad a los negocios, y puede expresarse a través del ciclo de vida de los procesos que se grafica en la figura siguiente:



Como muestra el gráfico, el ciclo de vida de los procesos consta de las siguientes actividades:

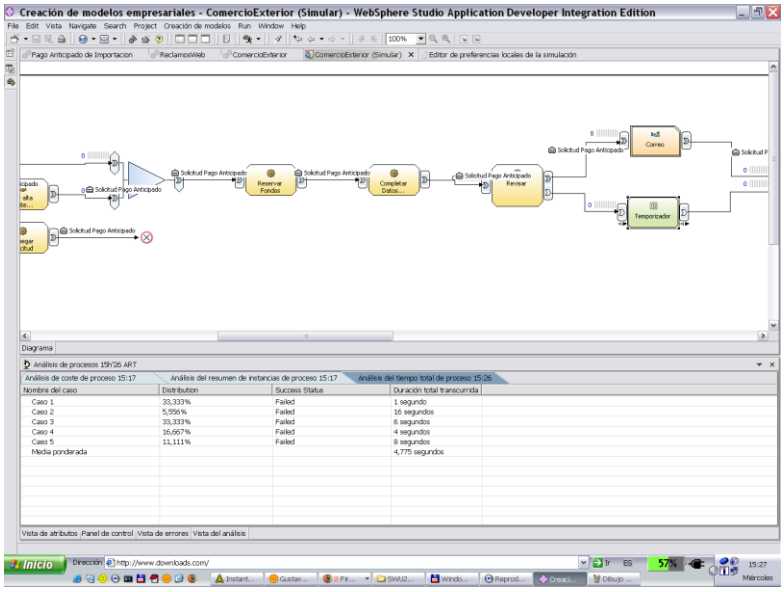
- **Modelar los procesos de negocio**: el analista de procesos o especialista en métodos y procedimientos aplica su conocimiento del negocio para crear gráficamente un modelo del proceso y simular en su estación de trabajo los resultados de su ejecución (tiempos, costos, ingresos, recursos)
- **Ensamblar los componentes necesarios**: lo cual implica completar el proceso modelado en el paso anterior con los elementos técnicos necesarios (componentes JEE, estructuras de datos, mensajes, etc) que posibiliten que aquel pueda efectivamente ejecutarse.
- **Poner en marcha** *("deploy"):* es decir, poner a ejecutar el proceso ensamblado util
- izando la infraestructura de software y hardware que sea necesaria, y que puede incluir elementos tales como: un motor de procesos, un ESB, etc.
- **Administrar los procesos**: o sea, monitorear su ejecución para poder corregir en tiempo real posibles desviaciones y situaciones de

excepción que puedan estar provocando por ejemplo demoras indeseables, y para poder evaluar los resultados de la ejecución contra las metas de negocio definidas.

La evaluación de los resultados de los procesos puede llevar en muchos casos a la decisión de rediseñarlos para poder cumplir mejor con las metas estratégicas del negocio, lo cual nos conduce nuevamente al modelado, cerrando de ese modo el ciclo.

Utilizando una herramienta de modelado los analistas de negocio y especialistas en procesos pueden, tomar como *input* los requerimientos definidos por los ejecutivos y los consultores en estrategia, para:

- Modelar en forma gráfica los procesos de negocios que resuelven esos requerimientos de la empresa
- Rediseñar rápidamente los procesos existentes para adecuarlos a nuevos requerimientos impuestos por las situaciones cambiantes del mercado
- Simular en la estación de desarrollo la ejecución de los procesos para evaluar sus tiempos, costos y resultados



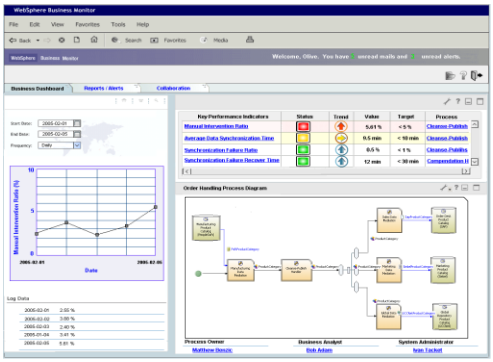
Los modelos de proceso que se crean con este tipo de herramientas pueden ser exportados por ejemplo en formato ***BPEL (Business Process Execution Language)***. BPEL es un lenguaje estándar para la descripción de procesos de negocio, basado en XML.

El modelo de proceso en formato BPEL puede ser tomado por una herramienta de desarrollo tal para que los desarrolladores lo ensamblen con los servicios convirtiéndolo de ese modo en una aplicación compuesta que puede ponerse a ejecutar bajo el control de un ***motor de procesos***. Vale la pena destacar que el mismo modelo del proceso se comparte

entre los analistas de negocio y los desarrolladores, ayudando así a alinear mejor a IT con el negocio.

Volviendo al ciclo integral de los procesos de negocio, vimos en el gráfico anterior que, una vez que el proceso está ejecutándose, puede ser monitoreado para:

- Mostrar en tiempo real los resultados de la ejecución en tableros de control personalizados
- Enviar alertas para poder tomar una acción a tiempo ante situaciones anormales
- Rastrear activamente eventos de negocios a lo largo de su ejecución a través de la cadena de valor



Esta actividad, que se suele denominar ***Business Activity Monitoring (BAM)***, permite monitorear la performance de los procesos, mejorar el control sobre el negocio, asignar recursos más eficientemente y tomar acciones (tales como balancear la carga de trabajo, suspender un proceso, etc.).

5. La infraestructura tecnológica de una SOA

Los procesos de negocio en el contexto de una SOA se ejecutan bajo el control de un motor o servidor de procesos, pero existen además diversos componentes de software que facilitan la creación de nuevos servicios, la integración de los procesos con servicios nuevos o ya existentes y la interacción de los servicios y procesos con las personas.

Estos componentes pueden clasificarse del siguiente modo:

- **Infraestructura de conectividad de servicios**, que se implementa, como ya se ha dicho, mediante un Enterprise Service Bus (ESB).

- **Servicios propiamente dichos**, que pueden ser brindados por:

- **Nuevos componentes aplicativos**, diseñados específicamente para SOA y desarrollados por ejemplo en JEE (Java Enterprise Edition) o en .NET, entre otras posibilidades.
- **Aplicativos preexistentes ("legacy"),** que pueden integrarse a SOA a través de adaptadores o conectores, o modificando el código mismo de los programas que las componen.
- **Servicios provistos por terceros.** En las comunicaciones entre empresas se ha utilizado históricamente una gran variedad de protocolos (EDI, RosettaNet, FTP, Web Services, etc). Para aislar a los aplicativos y procesos de la complejidad de manejar todos esos diferentes protocolos, y para manejar desde un lugar único cosas tales como la seguridad y la calidad del servicio, es conveniente contar con un *gateway*, que provea una interface de servicios a los aplicativos y procesos de la empresa y que se ocupe de manejar la comunicación con los diferentes proveedores de servicios externos.

- **Elementos que permitan componer servicios**, que incluyen:

  - **Servidor de procesos**: ya mencionado anteriormente.
  - **Portales web:** los portales web permiten crear aplicaciones compuestas integrando servicios en el navegador web *("integration on the glass")*. Los portales permiten además presentar de manera personalizada para cada tipo de usuario toda la información que cada usuario necesita para llevar a cabo sus tareas, incluyendo la lista de items de trabajo del motor de procesos. Los portales son por esta razón un elemento esencial para dotar a los procesos de negocio, en particular a aquellos en los que existen tareas humanas, de una gran eficiencia ya que cada persona dispondrá de la información necesaria en forma integrada y consolidada en un único lugar.

  - **Integración de la información:** conocida también como Federación de Datos, que permite acceder a múltiples fuentes diferentes de datos como si se tratase de una base de datos única y coherente, presentando además los resultados en la forma de servicios.

5. Conclusión

Hemos explicado los conceptos generales de la arquitectura orientada a servicios (SOA). SOA es, según el consenso general de la industria hoy, la mejor manera para conseguir la flexibilidad que las empresas necesitan hoy para poder adaptarse rápidamente a los cambios a un costo razonable.

SOA resuelve el problema de la integración transversal atacándolo en la raíz: reemplaza las aplicaciones monolíticas por un conjunto de servicios independientes y débilmente acoplados, de manera que los servicios actúan como verdaderas "cajas negras". Si un servicio cambiase, modificando algunas reglas a nivel del ESB puede lograrse que todo siga funcionando en forma transparente para los restantes servicios que no se verían afectadas en absoluto por el cambio.

En definitiva, de lo que se trata es de encapsular la implementación interna de cada servicio, publicando a la vez interfaces estándar y bien definidas para facilitar su reutilización. Si se le agrega a esto la facilidad de combinar los servicios individuales en aplicaciones compuestas que representan procesos de negocio, se puede entender porque esta arquitectura puede ofrecer la flexibilidad requerida a costos reducidos.

SOA (Service-Oriented Architecture) fue una base clave en la evolución de los sistemas distribuidos, especialmente en grandes organizaciones. Su idea principal: exponer funcionalidades como servicios reutilizables, desacoplados y orquestados para cumplir procesos de negocio.
Hoy SOA sigue vigente en ciertos entornos corporativos, pero fue parcialmente reemplazada o complementada (NO ELIMINADAS POR COMPLETO) por arquitecturas más modernas como:

- **Microservicios**
- **API**
- **Event-Driven Architecture (EDA)**

¿Actualmente Se sigue usando solamente ESB (Enterprise Service Bus)?
Depende del entorno.

- En empresas grandes con infraestructura SOA madura: sí, sigue habiendo ESBs como:
    - Mule ESB
    - WSO2 ESB
    - IBM Integration Bus (IIB)
    - Oracle Service Bus
- En arquitecturas modernas, el ESB fue reemplazado por:
    - API gateways (Kong, Apigee, AWS API Gateway)
    - Service meshes (Istio, Linkerd)
    - mensajería asincrónica (Kafka, RabbitMQ)

Conclusión: el rol del ESB cambió. Hoy se evita el "ESB pesado" (que centraliza toda la lógica) y se prefiere desacoplar e integrar mediante APIs y eventos.

¿Qué son los microservicios?

Los microservicios son una forma de diseñar software como un conjunto de servicios pequeños, independientes y especializados, que se comunican entre sí a través de redes (normalmente usando APIs).

Caracteristicas:

- Independencia: Cada microservicio puede desarrollarse, desplegarse y escalarse por separado.
- Responsabilidad única: Cada uno cumple una función específica (por ejemplo: pagos, usuarios, inventario).
- Tecnología libre: Cada servicio puede usar diferentes lenguajes, frameworks y bases de datos.
- Comunicación por red: Generalmente usando HTTP (REST), mensajería o gRPC.

Infraestructura básica para crear y publicar una API

1. Lenguaje y Framework

Primero necesitás programarla. Algunos ejemplos:

- Node.js + Express
- Python + Flask o FastAPI
- Java + Spring Boot
- .NET Core
- Ruby on Rails (API mode)

2. Servidor o entorno de ejecución

Donde va a correr tu API:

- Localmente (durante desarrollo).
- Luego: en un servidor real (ej. VPS, cloud) o plataformas de despliegue: ejemplo…Amazon AWS, Azure, Google Cloud, Docker

3. Base de datos

- Relacional: PostgreSQL, MySQL
- No relacional: MongoDB, Firebase
- Puede estar en el mismo servidor o en uno aparte

4. Control de versiones

- plataformas como GitHub o GitLab para controlar cambios, colaborar y desplegar.

5. Pruebas y documentación

- Usá herramientas como:
  - Postman o Insomnia para testear
  - Swagger / OpenAPI para documentar tu API

6. Seguridad y control de acceso

- Autenticación y autorización si se requiere

Qué es una API?

Una API (Application Programming Interface) es una puerta de entrada a un sistema. Permite que otros programas se conecten, pidan información o realicen acciones sin necesidad de ver cómo funciona todo por dentro.

Ejemplo real y simple:
Imaginá esto:
Tenés una app de pretamos en el celular. La app pide los datos a una API, ejemplo VERAZ (como lo vimos en clase).

1. La app dice: "API, decime si este CUIT, tiene deuda".
2. La API del VERAZ responde: "cuit con DEUDA".

Partes básicas de una API

1. Cliente (el que pide)
   Puede ser una app, página web o cualquiera desde un navegador.
2. Servidor (el que responde)
   Es donde está la API, con toda la lógica.
3. Endpoints , son como puertas numeradas. Ejemplo:
   - GET /productos
   - POST /usuarios
   - PUT /tareas/1

¿Cómo funciona una API?

Supongamos una API para una tienda de mates online:
Paso 1 – El cliente hace una petición:
GET https://mitienda.com/api/productos
//Esto significa: "Dame la lista de productos".

Paso 2 – El servidor responde con un mensaje en JSON (podría ser también un xml)
Json

```
[
  { "id": 1, "nombre": "fideos", "precio": 2500 },
  { "id": 2, "nombre": "azucar", "precio": 1800 }
]
```

Tipos de acciones (métodos HTTP)

Acción	Método	Qué hace
Leer	GET	Pedir datos
Crear	POST	Enviar nuevos datos
Modificar	PUT o PATCH	Cambiar datos existentes
Borrar	DELETE	Eliminar datos

Swagger es una herramienta (y un conjunto de especificaciones) que te permite diseñar, documentar, probar y consumir APIs REST de forma fácil y visual.

¿Qué hace Swagger?

1. Documentación interactiva de tu API

Genera una página web donde ves todos los endpoints de tu API (GET, POST, PUT, DELETE, etc.).

- Cada endpoint incluye sus parámetros, ejemplos de respuesta y un botón de "Probar" para enviar peticiones directamente desde el navegador.

2. Comunicación clara entre desarrolladores
   - Swagger actúa como contrato entre el backend y el frontend: todos saben qué hace cada endpoint, qué datos espera y qué devuelve.
3. Valida y prueba automáticamente
   - Podés usar Swagger UI para probar tu API sin necesidad de usar Postman o curl.
4. Define tu API con un archivo YAML o JSON
   - Usás un archivo llamado swagger.yaml o swagger.json para describir tu API (endpoints, parámetros, modelos, etc.).